

LLM Orchestration

Building a project-agnostic agentic core, module by module

Study notes · Python / Pydantic / LiteLLM / asyncio / OpenTelemetry

Overview

These notes reconstruct a small, framework-agnostic *agentic core*: the thin layer every LLM call in a system flows through. The one idea the whole design hangs off is:

The one idea. Every model call passes through a single chokepoint — the gateway. That one well-instrumented path is what lets reliability, latency, and cost be guaranteed *centrally*, instead of scattering retries, timeouts, and cost logic across the codebase.

The spine of the system, from the outside in:

request → **orchestration** → **agent(s)** → **gateway** → router (LiteLLM) → provider(s)

What is covered. Each section is one module of that build, in order: the *architecture*, the typed *schemas*, the error *taxonomy*, the *gateway* keystone, *observability*, the *agent* abstraction, the *orchestration* layer (with a deep dive on concurrency correctness and on running at scale), *guardrails*, client-side *tools*, and the *eval* harness. Two topics — the context-manager mechanics behind observability, and the atomicity/idempotency of the concurrent merge — are treated at extra depth. The running theme throughout: *put each guarantee in one place, and let structure (a type, a context manager, a lock, a **WHERE** clause) enforce it* rather than hoping call sites behave.

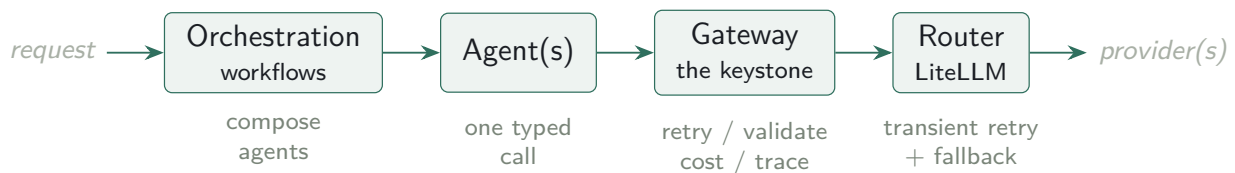
Contents

Overview	1
1 Architecture: the layers	2
2 <code>schemas.py</code> — contracts and carrier types	3
2.1 Why a dataclass, not a BaseModel	3
2.2 frozen, slots, and Generic[T]	3
2.3 What BaseModel actually is (the boundary)	4
2.4 The mutable-default footgun	4
3 <code>errors.py</code> — the failure taxonomy	4
3.1 Classifying a raw provider error	5
3.2 HTTP status families: 4xx vs 5xx	5
4 <code>gateway.py</code> — the keystone	5
4.1 RouterLike — depend on a shape, not on LiteLLM	5
4.2 Two modes: production and test	6
4.3 <code>complete()</code> — an async instance method	6
4.4 Structured output: <code>response_model</code>	6
4.5 Why <code>async/await</code> everywhere	7
4.6 The re-prompt loop	7
4.7 Testing: a fake router	8
4.8 <code>params</code> vs <code>**opts</code>	8

5	observability.py — one span per call	8
5.1	The building blocks	8
5.2	The span wrapper	9
5.3	Using it in the gateway	9
5.4	Enter / exit order	9
6	agents/ — the agent abstraction	10
6.1	system_prompt vs user_template	10
6.2	Output contracts: Literal and Field	10
7	orchestration/runner.py — composition	11
7.1	Concurrency and correctness: the merge race	11
7.2	Acting on the first result: other asyncio tools	13
7.3	Durable execution at scale	13
8	guardrails/io_guards.py	13
9	agents/tool_loop.py — client-side tools	14
9.1	Agents as tools	15
10	eval/harness.py	15
	Closing	16

1 Architecture: the layers

A request enters at an **API endpoint** and descends through the layers, each adding one concern:



The spine: each layer wraps the one below it and adds a single concern.

- **Orchestration / workflows** — composes agents (sequential, concurrent, conditional). Project-specific files, e.g. `parse_workout.py`, `weekly_reports.py`, `recommend.py`.
- **Agents** — one typed, validated unit of LLM work each.
- **Gateway** — the keystone: the single client every call passes through.
- **Router (LiteLLM)** — one OpenAI-format interface to 100+ providers, owning transient retries and fallback.

The API layer is thin: a route hands off to a workflow, e.g.

Block 1 ■ API layer — a thin entry point

```

1 @app.post("/parse-workout")
2 async def parse_workout_endpoint(req):
3     return await parse_workout(req.text)    # delegate straight to the workflow
  
```

MCP. MCP (Model Context Protocol) is the emerging standard for the agent↔tools boundary. It is a protocol to know by name; this core does not adopt it, but the tool layer

(§9) is where it would slot in.

2 schemas.py — contracts and carrier types

There are *two* kinds of contract, and only one belongs in this file.

- **Agent contracts** — a given agent’s input/output shape. These are *project-specific* and live *next to their agents*. Plain `pydantic.BaseModel` subclasses.
- **Carrier types** — what flows through *every* call regardless of the project. These live here. There are two: `TokenUsage` (token counts) and `Completion` (the gateway’s return value).

`Completion[T]` is everything one trip through the gateway produced:

Block 2 ■ schemas.py — the carrier types

```

1 @dataclass(frozen=True, slots=True)
2 class TokenUsage:
3     prompt_tokens: int = 0
4     completion_tokens: int = 0
5     total_tokens: int = 0
6 T = TypeVar("T", bound=BaseModel) # only Pydantic models may be T
7 @dataclass(frozen=True, slots=True)
8 class Completion(Generic[T]):
9     text: str # raw assistant output; source of truth
10    model: str # concrete model that served (post-fallback)
11    parsed: T | None = None # validated structured output, or None (free text)
12    usage: TokenUsage | None = None # observability + cost sweeps read these three
13    cost_usd: float | None = None
14    latency_ms: float | None = None
15    reprompt_attempts: int = 0 # gateway re-prompts (NOT transient retries)
16    extra: dict = field(default_factory=dict) # provider extras, no contract widening

```

Code blocks are numbered; lines are cited as “(Block N, l.M)” — e.g. this is Block 2.

2.1 Why a dataclass, not a BaseModel

Pydantic’s job is *validation at a boundary*. But `Completion` wraps an *already-validated* payload (`parsed`, Block 2, l.11, is already a validated model), so re-validating it would be wasted work on the hot path. So the carriers are plain frozen dataclasses — records, not boundaries.

BaseModel	dataclass
enforces field types; <i>raises</i> <code>ValidationError</code> on bad input	just stores what it is given; no validation
use at a <i>boundary</i> (untrusted input)	use for an <i>internal record</i>

2.2 frozen, slots, and Generic[T]

- `frozen=True` (Block 2, l.1, 7) — immutable, because a completed call is a *fact*, not mutable state; a downstream tool must not change fields after creation.
- `slots=True` (Block 2, l.1, 7) — memory/attribute-speed win, and blocks typo’d attributes.
- `Generic[T]` (Block 2, l.8) — a typed call yields a `Completion[Invoice]` whose `.parsed` (l.11) is statically `Invoice | None` when the caller passes `response_model=Invoice`. `T` is bound to `BaseModel` (l.6), so only Pydantic models can fill it.

Mutable vs immutable. **Mutable** = can change after creation (e.g. a `list`). **Immutable** = cannot (e.g. a `tuple`). `Completion` and `TokenUsage` are immutable because a completed call is a fact — freezing it makes a whole class of bugs impossible by construction.

2.3 What BaseModel actually is (the boundary)

A `BaseModel` does not trust its input. At class-definition time it *compiles a validator* from the field annotations; constructing one runs that validator and raises `ValidationError` on bad input.

Block 3 ■ An agent output contract lives next to its agent

```
1 class Invoice(BaseModel):
2     total_amount: float
3     currency: str
4     # reads the fields, compiles a validator, checks types against the model,
5     # raises ValidationError on bad input -- exactly what the gateway runs.
```

The gateway turns raw LLM text into a validated model with the classmethod `model_validate_json` — the single line where untrusted output becomes trusted data or triggers a re-prompt.

2.4 The mutable-default footgun

A default value is created *once*, at definition time, and shared across all instances. So a *mutable* default is shared — which is exactly why `extra` (Block 2, l.16) uses `field(default_factory=dict)`, not `= {}`:

Block 4 ■ Why extra uses a default_factory

```
1 @dataclass
2 class Bad:
3     extra: dict = {}           # <-- one dict, shared across ALL instances
4     a = Bad(); b = Bad()
5     a.extra["x"] = 1
6     print(b.extra)             # {'x': 1} -- b sees a's mutation (same dict!)
7
8     # fix: a fresh dict per instance
9     extra: dict = field(default_factory=dict)
```

3 errors.py — the failure taxonomy

The one distinction. **Reliability is mostly error classification** — knowing which failures to retry, which to re-prompt, and which to fail on. Two retry-shaped jobs exist, each owned by exactly one layer, so they compose instead of multiplying.

Category	Owner	Meaning / action
<code>TransientError</code>	Router	network-level (timeout, rate-limit, 5xx). The Router retries; the gateway never does.
<code>PermanentError</code>	—	auth, bad request, content policy, unknown model. Fails again identically → fail fast.
<code>MalformedOutputError</code>	Gateway	the model <i>replied</i> , but the content didn't satisfy the Pydantic schema → the gateway re-prompts.
<code>AllModelsFailedError</code>	Gateway	the Router exhausted its retries <i>and</i> every fallback → terminal.

The malformed case is subtle: to the Router that call *succeeded* (valid HTTP response with text). Only the gateway holds the schema, so only the gateway can re-prompt. The Router has no concept of our schema.

3.1 Classifying a raw provider error

`classify_provider_error` maps a raw Router exception onto the taxonomy. It is called at one point — when `Router.acompletion` raises:

Block 5 ■ gateway.py – surfacing a terminal Router failure

```

1  try:
2      response = await router.acompletion(model=model, messages=messages, **opts)
3  except Exception as exc:
4      classified = classify_provider_error(exc)
5      if isinstance(classified, PermanentError):
6          raise classified from exc                # would never have worked
7      raise AllModelsFailedError(
8          "Router exhausted retries and fallbacks",
9          attempted=[model, *fallbacks],
10         ) from exc                                # transient-rooted, terminal

```

`raise ... from exc`. The `from exc` chains the original provider exception onto the new one: the traceback shows *both* the clean, classified error *and* the original root cause. You surface a meaningful error without destroying the forensic trail. `attempted` is a list of model names (a roster to act on); the *why* lives in `__cause__`.

3.2 HTTP status families: 4xx vs 5xx

Range	Whose fault	Retry?
2xx	success	—
4xx	<i>you</i> (client) — 401 auth, 400 bad request, 404 not found	usually no (permanent); <i>except</i> 429 rate-limit, treated as transient
5xx	<i>them</i> (server) — 500 internal, 502 bad gateway, 503 unavailable, 504 timeout	usually yes (transient)

Mental split: 4xx = you broke it = permanent; 5xx = they broke it = transient = the Router's retry domain.

4 gateway.py — the keystone

4.1 RouterLike — depend on a shape, not on LiteLLM

The gateway depends on a one-method *Protocol*, not on `litellm.Router` concretely:

Block 6 ■ gateway.py – the dependency the gateway actually needs

```

1  class RouterLike(Protocol):
2      async def acompletion(self, *, model, messages, **kwargs): ...

```

A `Protocol` is *structural* typing: *anything* with an `acompletion` method satisfies it, no inheritance required. If an object has that method with that signature, the type checker treats it as a

RouterLike. This buys three things: **testability** (inject a fake), a **narrow** documented dependency, and **swappability**.

4.2 Two modes: production and test

One constructor, two modes, via optional dependency injection:

Block 7 ■ gateway.py – inject a fake, or build a real router

```
1 self._router = router or self._build_router(registry, fallbacks)
2
3 gw = Gateway() # Mode 1 (production): builds a real LiteLLM Router
4 gw = Gateway(router=FakeRouter([...])) # Mode 2 (test): injected, short-circuits the 'or'
```

The or idiom. A `or B` returns A if it is *truthy*, else B. A router object is *truthy*, and the default `None` is *falsy* — so a provided router wins and `_build_router` (which lazily imports the heavy `litellm.Router`) is never called. Config code uses `x if x is not None else default` instead, where an empty-but-valid value must survive.

`_build_router` calls `build_router_kwargs`, the one seam between config and the Router. It compiles the registry into the exact dict LiteLLM wants:

Block 8 ■ config.py -> Router(**kwargs)

```
1 {
2     "model_list": [
3         {"model_name": alias,
4          "litellm_params": {"model": "openrouter/openai/gpt-4o-mini",
5                             **params, "api_key": key, "rpm": rpm, "tpm": tpm}},
6     ],
7     "num_retries": settings.num_retries, # Router's transient-retry domain
8     "timeout": settings.request_timeout_s,
9     "fallbacks": [{"smart": ["fast"]}],
10 }
```

Generation knobs (`temperature`, `max_tokens`) go in `params`; rate caps go in the typed `rpm/tpm` fields (siblings of `params`, omitted when `None`).

4.3 complete() — an async instance method

`complete` is an **async instance method**: it uses instance state (`self._router`, `self._settings`), so it is *not* a `@staticmethod` (no `self`) nor a `@classmethod` (`cls` first). Parameters: `messages`, `model`, `response_model`, and `**opts`. The bare `*` in signatures makes following parameters keyword-only, so call sites are self-documenting and un-transposable.

`model` is a registry *alias* ("`fast`"/"`smart`"); the Router maps it to a concrete provider string.

4.4 Structured output: response_model

When the caller asks for structured output, the gateway appends a *schema instruction* to the prompt — provider-agnostic on purpose, so the re-prompt loop is the real guarantee:

Block 9 ■ gateway.py – the schema instruction

```
1 schema = json.dumps(response_model.model_json_schema()) # BM → JSON Schema string
2 base = [*messages, {"role": "system",
3                  "content": f"Respond with JSON matching:\n{schema}"}]
```

4.5 Why async/await everywhere

await = pause at slow I/O. An LLM call is **I/O-bound**: almost all its wall-clock time is spent *waiting* on the network. **await** means “this is a slow I/O point — pause me here, run other tasks, wake me when the result is ready.” **async** marks a function pausable. It is *contagious*: because the bottom call (**acompletion**) is awaited, every layer above — gateway, retry loop, agents, orchestration — must be **async** too.

The payoff is concurrency: two independent calls can wait *at the same time*.

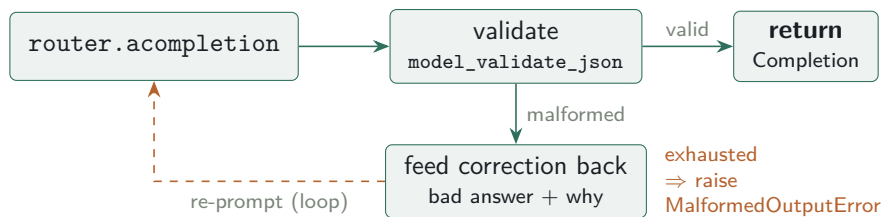
Block 10 ■ Concurrent calls overlap their network waits

```
1 results = await asyncio.gather(gw.complete(msgs_a), gw.complete(msgs_b))
2 # both requests in flight at once → total time ~ the slower one, NOT the sum
```

Async concurrency is *not* multi-core parallelism: it is one thread interleaving tasks at their **await** points, which is exactly right for I/O-bound work.

4.6 The re-prompt loop

The gateway owns *one* retry-shaped job — re-prompting on malformed output — driven by **tenacity**’s **AsyncRetrying** (Block 11, l.1; the **async** variant, because the retried body **awaits** the router). For each attempt: run the router (l.7), extract the text (l.8), validate (l.11); on failure, feed the model its own bad answer plus the error (l.13) and loop.



Only **MalformedOutputError** loops; a provider error propagates out to be classified.

Block 11 ■ gateway.py – validate, or re-prompt

```
1 async for attempt in AsyncRetrying(
2     stop=stop_after_attempt(1 + self._max_reprompts),
3     retry=retry_if_exception_type(MalformedOutputError), # ONLY this error retries
4     reraise=True,
5 ):
6     with attempt:
7         response = await self._router.acompletion(model=model, messages=base + feedback, **
8             ↪ opts)
9         text = self._extract_text(response)
10        if response_model is not None:
11            try:
12                parsed = response_model.model_validate_json(self._strip_fences(text))
13            except (ValidationError, ValueError) as exc:
14                feedback = self._correction(text, exc) # bad answer + why it failed
15                raise MalformedOutputError(..., raw_output=text, validation_error=exc)
16        return self._build_completion(...) # success
```

Why raise is the “retry” signal. **raise MalformedOutputError** (Block 11, l.14) instantly abandons the current attempt (the **return** on l.15 is never reached) and exits the **with attempt:** block (l.6). **tenacity** sees it matches the predicate (l.3) and loops — or, when attempts are exhausted, **reraise=True** (l.4) surfaces the error. A raw *provider* exception does

not match the predicate, so it propagates straight out to the `except` blocks and is classified there. Transient network retries are the Router’s job; the gateway never retries them.

A raised exception travels a *separate channel* from the return value, so a function annotated `-> Completion` can still raise on the failure path — the annotation governs only what is *returned*. The single thing that would violate it is falling off the end (implicit `return None`); an `AssertionError` backstop at the bottom plugs that.

4.7 Testing: a fake router

Because the gateway depends on `RouterLike`, tests inject a scripted fake (no network, no keys). A `SimpleNamespace` mimics the response shape by attribute (*duck typing*):

Block 12 ■ conftest.py – an injectable fake

```

1 class FakeRouter:                                # does NOT inherit RouterLike (structural match)
2     def __init__(self, behaviours): self._behaviours = list(behaviours); self.calls = []
3     async def acompletion(self, *, model, messages, **kwargs):
4         self.calls.append({"model": model, "messages": messages})
5         b = self._behaviours.pop(0)
6         if isinstance(b, Exception): raise b
7         return b

```

4.8 params vs **opts

Sampling knobs live in two places: a deployment’s `params` (the per-alias *default*, in `config.py`) and per-call `**opts`. **Per-call opts win** — `{**params, **opts}`. Put stable defaults in `params`; reserve `**opts` for a one-off deviation.

5 observability.py — one span per call

The observability module wraps *one span* around every gateway call. Because every call already flows through the gateway, instrumenting that single place gives complete, uniform coverage — latency, tokens, cost, model, errors — for free. It is built with `@contextmanager`, so it rests on four Python features that compose.

5.1 The building blocks

`with ... as` is guaranteed setup/teardown: `with cm as y:` calls `cm.__enter__()` on entry (binding `y`), runs the block, then *always* calls `cm.__exit__(...)` on exit — success or exception. The object after `with` must be a *context manager* (has `__enter__`/`__exit__`).

A **generator** is a function with `yield`: `yield` makes it *pausable* — hand back a value, freeze, resume later. Its shape is before-yield / yielded-value / after-yield.

A **decorator** takes a function and returns an enhanced one. `@contextmanager` turns a one-yield generator into a context manager: *before yield* → `__enter__`, *yielded value* → the `as` variable, *after yield* → `__exit__`.

The one-yield rule. A `@contextmanager` generator must reach **exactly one** `yield` on every path. Code before it is setup; code after it is teardown (and it receives any exception thrown back in at the yield). Without the decorator, `with completion_span(...)` would raise `TypeError: 'generator' object does not support the context manager protocol` — the decorator manufactures exactly what `with` consumes.

5.2 The span wrapper

Block 13 ■ observability.py – the span context manager

```

1 @contextmanager
2 def completion_span(*, model, response_model=None):
3     with _TRACER.start_as_current_span(_SPAN_NAME) as span:      # OTEL's OWN context manager
4         span.set_attribute("gen_ai.request.model", model)        # decorate the span
5         if response_model is not None:
6             span.set_attribute("agentic.response_model", response_model)
7         try:
8             yield span                                           # <-- setup done; hand span to the caller
9         except BaseException as exc:
10            span.record_exception(exc)                            # observe failures too
11            span.set_status(Status(StatusCode.ERROR, type(exc).__name__))
12            span.set_attribute("agentic.error.type", type(exc).__name__)
13            raise                                                 # never swallow -- propagate unchanged

```

There are *two nested* context managers: OTEL's inner one (`start_as_current_span`, Block 13, l.3) owns the raw span lifecycle (start/end); the outer `@contextmanager` (l.1) adds request attributes (l.4) and error-recording (ll.10–13), and re-yields the span upward (`yield span`, l.8). On an exception the exception is thrown *back into* the generator at l.8, caught by the `except` (l.9), recorded, and re-raised (l.13) — observed, never swallowed.

5.3 Using it in the gateway

Block 14 ■ gateway.py – `complete()` wraps the whole call in one span

```

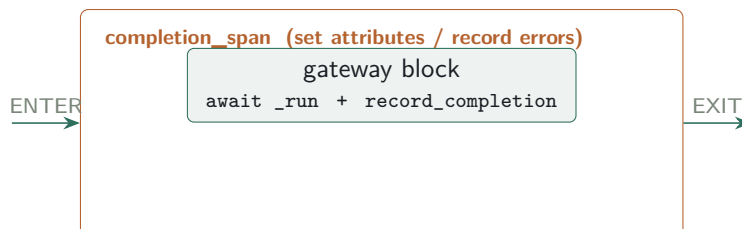
1 with completion_span(model=model,
2                      response_model=response_model.__name__ if response_model else None) as
3     ↪ span:
4     completion = await self._run(messages, model=model,
5                                response_model=response_model, **opts)
6     record_completion(span, completion, alias=model)  # plain fn, writes to the span
7     return completion

```

`record_completion` (Block 14, l.5) is *not* a context manager — it is an ordinary function called *inside* the `with` block (opened on l.1), stamping result metrics (tokens, cost, latency) onto the yielded `span`. Only the thing *after* `with` needs the decorator.

5.4 Enter / exit order

Nested managers unwind in mirror order (outer enters first, exits last). The raw span is created as late as possible and closed as the innermost thing, so it is never left open:



ENTER: your cm → OTEL cm → set attributes → yield. EXIT (mirror): resume → record error? → OTEL ends span → your cm returns.

The exit fires only after the *whole* block finishes (after both the call *and* `record_completion`), so metrics are written while the span is open, then it closes. On failure, `_run` raises, `record_completion` is skipped, and the `except` records the error before OTEL closes the span.

No-op by default. The module talks to the OpenTelemetry *API*, not a backend. Until an app calls `configure`, the global tracer is a non-recording no-op: spans run but reach no network — which keeps the test suite offline while the gateway is instrumented unconditionally. `configure` points spans at Logfire (with a token), a local console, or a backend like Jaeger (via OTLP). OTel itself needs no key — it is a standard, not a service.

Observability precedes optimisation. A threshold like “9 seconds is too slow” ^{WHITEBOARD} only means something because the latency distribution was measured first — and it was measured because one span wraps the chokepoint, not because tracing was sprinkled everywhere.

6 agents/ — the agent abstraction

An agent is a typed, validated, single-responsibility LLM call. It owns exactly three things:

1. *How to turn typed input into messages* (`render_messages` / the templates).
2. *Which typed output it expects* (`output_model`, enforced by the gateway).
3. *Which model alias it runs on* (`model`).

An agent *never* talks to a provider; it only ever calls `gateway.complete`. Its `run` returns the full `Completion` — `.parsed` is the typed output, and the cost/latency/trace metadata rides along so orchestration keeps visibility.

6.1 system_prompt vs user_template

The `system_prompt` is fixed instructions, identical on every call, used verbatim (*Block 15, l.3* below). The `user_template` is a *fill-in-the-blanks* string (*l.6*) whose `{placeholders}` are substituted with this call’s input:

Block 15 ■ base.py – render typed input into messages

```
1 def render_messages(self, inp, *, history=None):
2     return [
3         {"role": "system", "content": self.system_prompt},           # fixed, verbatim
4         *(history or []),                                           # prior turns, if any
5         {"role": "user",
6          "content": self.user_template.format(**inp.model_dump())}, # {fields} filled in
7     ]
```

`inp.model_dump()` (*Block 15, l.6*) turns the validated model into a plain dict keyed by *field names*; `**` unpacks it into keyword arguments so `.format` fills each `{name}`. Every placeholder must match an input field — so editing the template and the input model are coupled edits. The optional `history` (*l.4*) threads prior turns between the system prompt and the new user message.

6.2 Output contracts: Literal and Field

Block 16 ■ examples.py – a tight output contract

```
1 class Sentiment(BaseModel):
2     label: Literal["positive", "negative", "neutral"] # closed set of allowed values
3     confidence: float = Field(ge=0.0, le=1.0)         # numeric range constraint
```

`Literal[...]` constrains `label` to *exactly* those three strings (an inline enum); `Field(...)` adds constraints/metadata — numeric (`ge`, `le`, `gt`, `lt`, `multiple_of`), string (`min_length`, `max_length`, `pattern`), defaults (`default_factory`), and docs (`description`, which flows into the schema instruction the model sees). Every constraint is one more rule the gateway enforces at the boundary

and re-prompts on. For disjoint ranges (“0–1 or 2–3”), a single `Field` cannot AND them — use a `@field_validator` or a union of constrained types.

7 orchestration/runner.py — composition

Three composition primitives, all generic over the state type:

- `run_sequential` — in order; step N sees what $N-1$ wrote to state.
- `run_concurrent` — at once (`asyncio.gather`); results in step order.
- `run_conditional` — branch on a predicate.

Each takes two arguments: a shared `state` and a sequence of `steps`, where a `step` is `async (state) -> result:`

Block 17 ■ runner.py – the vocabulary

```

1 StateT = TypeVar("StateT")
2 Step = Callable[[StateT], Awaitable[ResultT]] # a unit of work over the shared state
3 @dataclass
4 class State:                                # a tiny mutable bag threaded between steps
5     data: dict[str, Any] = field(default_factory=dict)
6     def __getitem__(self, k): return self.data[k]
7     def __setitem__(self, k, v): self.data[k] = v

```

The runner runs *steps*, not *agents*. `run_sequential` imports no `Agent` and knows nothing about the gateway. A workflow *wraps* an agent into a step and hands it over — `lambda s: agent.run(s["text"])`. So `await step(state)` is what invokes the agent. This “compose steps, not agents” genericity is what keeps orchestration a thin, dependency-free, swappable layer. `Step` (Block 17, l.2) is the injected unit; `State` (ll.3–7) is a convenience default a workflow may swap for its own typed model.

Block 18 ■ workflow.py (consumer) – wrap agents, run concurrently, merge later

```

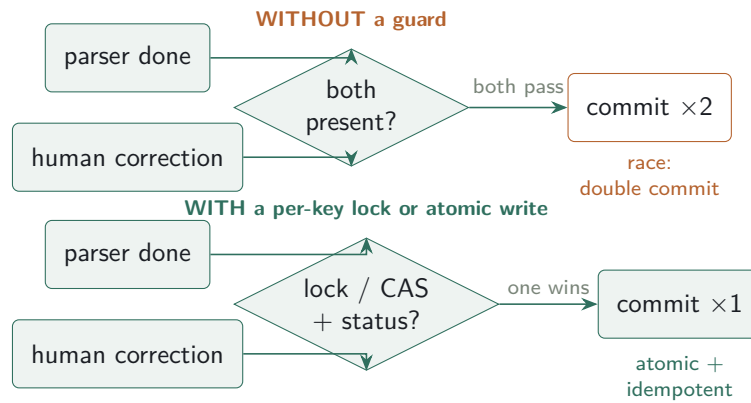
1 steps = [lambda s: interpret_agent.run(s["text"]),
2           lambda s: parse_agent.run(s["text"])]
3 results = await run_concurrent(state, steps) # each RETURNS a value, no shared write
4 state["interpreted"], state["parsed"] = results # merge AFTER, single-threaded, race-free

```

Sequential vs concurrent state discipline. Sequential steps communicate via *shared mutation* of `state` (safe, because ordered). Concurrent steps must treat `state` as *read-only* and communicate via *return values* — `gather` collects them in step order, and the caller merges them afterwards. Several concurrent steps writing the shared bag at once would race.

7.1 Concurrency and correctness: the merge race

Whenever several concurrent events must “finish” one unit of work **exactly once**, a race appears. Two handlers both read the state, both see “everything present, not yet done,” and *both* commit — double commit, duplicate notification, corrupted row.



Fan-in barrier: mutual exclusion over check-and-commit (so two don't both commit) *plus* a status guard (idempotency).

Two guarantees, not one. Atomicity / mutual exclusion stops *two* handlers from both deciding “ready.” **Idempotency** (a status guard) makes a *repeated* commit harmless. Neither alone suffices: idempotency does not stop double *side effects* (“charge the card” twice); atomicity does not cover at-least-once retries.

Races live at await points. In one event loop a coroutine yields *only* at an `await`; synchronous code between awaits is already atomic. A critical section needs protection precisely when it contains an `await` *between the check and the commit* — that yield is the interleaving point.

Strategy 1 — keep the await inside, add a per-key lock:

Block 19 ■ `asyncio.Lock` scoped per key

```

1 locks: dict[str, asyncio.Lock] = defaultdict(asyncio.Lock)
2 async def on_event(key, name, result):
3     state[key].results[name] = result
4     async with locks[key]:                                # critical section: one at a time
5         s = state[key]
6         if s.status == "done": return                    # idempotency guard: someone won
7         if all(a in s.results for a in REQUIRED):
8             s.status = "done"
9             await commit(merge(s.results))                # the once-only action

```

Scope the lock *per key*, not globally, so only events for the *same* unit contend. **Strategy 2** removes the only `await` from the decision (do the synchronous claim, then the I/O after), needing no lock. When the state lives behind I/O (a database), the `await` is *un-removable* and an `asyncio.Lock` only works within one process — so push atomicity into the store:

Block 20 ■ Store-level atomicity – an atomic conditional write

```

1 async def try_mark_ready(row_id, merged) → bool:
2     result = await session.execute(
3         update(Row).where(Row.id == row_id, Row.status != "ready") # only if NOT ready
4         .values(status="ready", merged_result=merged))
5     await session.commit()
6     return result.rowcount == 1                                # True only for the caller that won

```

The conditional `UPDATE ... WHERE status != 'ready'` is *both* the atomicity fix (the DB serialises it) and the idempotency guard (the `WHERE` clause); side effects are gated on `won`.

Where state lives	Mechanism
single process, in memory	<code>asyncio.Lock</code> per key, or a lock-free synchronous critical section. Fast, but non-durable and per-process.
behind I/O, or cross-process / cross-restart	atomicity in the shared store: an atomic conditional write, a DB transaction, or a distributed lock (Redis).

7.2 Acting on the first result: other asyncio tools

`gather` waits for *all* (results in order); to act on the first finisher instead, use `as_completed` (handle each result the moment it is ready — results in *completion* order, losing positional alignment), `wait(FIRST_COMPLETED)` (react to the first, manage the rest), or `create_task` (start work in the background, await it later). *Streaming* is a different axis: the first *tokens* of one call as it generates, not the first *call* to finish.

7.3 Durable execution at scale

The in-memory runner is right for workflows that fit one request. For a human-in-the-loop step — an *unbounded* wait (seconds to days) — the workflow must **suspend completely to durable storage and resume by id**, possibly on another worker, possibly after a restart. State in RAM (the runner's `State`, a parked coroutine) dies on restart.

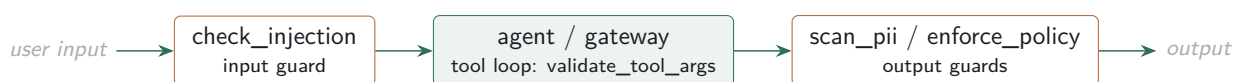
Two tiers of orchestration. In-process: `runner.py` — lightweight, ephemeral. **Durable:** a workflow *engine* — Temporal / AWS Step Functions / Azure Durable Functions / LangGraph. They are *alternatives* at the same layer; you pick one per workflow, and both call the same stateless agents below. `Celery/Redis` is a tier lower — a *task queue* for durable background jobs, but no checkpointing and no human-gate, so the state machine is still yours.

Hand-rolled durability = a DB for checkpointed state + a durable queue + a resume endpoint + idempotency keys (the transactional outbox and compare-and-set of the previous section). Adopt an engine when that machinery grows into a graph executor. With Temporal, an *activity* is the side-effecting agent call, a *workflow* is durable/replayable orchestration code, and a *signal* delivers the human's decision exactly once.

Three retry layers, three failure classes. The **Router** retries *transient* network blips (fast, in-process). The **gateway** re-prompts *malformed* output. The **engine** re-runs the *whole step* on *terminal* failure (durable, later). They nest — the Router exhausts first; the engine re-runs only if it still failed — so tune the engine with backoff + capped retries + idempotency to avoid multiplication.

8 guardrails/io_guards.py

Guardrails treat the model as untrusted: check what goes in, check what comes out. They are explicit functions the *workflow* calls at boundaries — the gateway stays about reliability, guardrails about policy.



Guards bracket the model call: input guard before, arg-validation inside the loop, output guards after.

- `check_injection` — *input* guard: a heuristic (regex) prompt-injection scan, run before the model sees untrusted text (OWASP LLM01). Honestly a first line, not a guarantee; `check_injection_llm`

is a stronger opt-in LLM-judge that routes through the gateway.

- `validate_tool_args` — validates model-proposed tool arguments against a Pydantic model *before* they execute (OWASP LLM06).
- `scan_pii / enforce_policy` — *output* guards (LLM02 / LLM05).

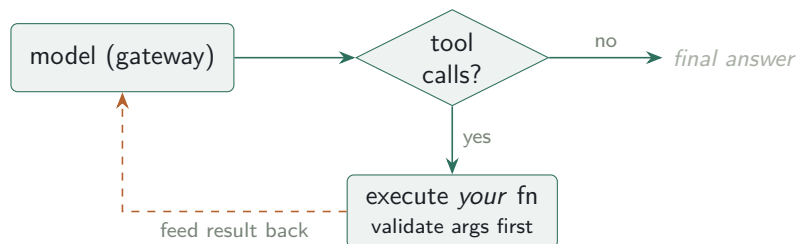
Block 21 ■ Guards at the boundary – fail-closed

```
1 check_injection(user_text).raise_if_failed()    # input boundary, before the model
2 # ... inside the tool loop, between "model asks" and "your code runs":
3 args = validate_tool_args(WeatherArgs, call.raw_args) # typed, or GuardrailError
```

Each guard returns an inspectable `GuardResult`; `.raise_if_failed()` is *fail-closed* — it raises `GuardrailError` (which propagates and stops the request). Guards live in the *workflow/consumer*, bracketing the agent call.

9 agents/tool_loop.py — client-side tools

The model only asks; you execute. A language model cannot run anything — it emits a structured *request* (“call `get_weather(city='Paris')`”). For a **client-side** tool, *your* code executes the function and feeds the result back. The **loop** is: call → detect tool calls → validate args → execute → feed result back → repeat, until the model returns a final answer. For a **server-side** tool (web search) the *provider* runs it inside one call — no loop for you.



The model only asks; the loop validates args, runs your function, feeds the result back, and calls again — until a final answer.

Block 22 ■ tool_loop.py – the call/execute/feed-back loop

```
1 while True:
2     resp = await gateway.complete(messages, tools=tools) # tools flow through **opts
3     if not resp.tool_calls:
4         break # final answer → done
5     messages.append(assistant_tool_message(resp))
6     for call in resp.tool_calls: # parallel tool calls, keyed by id
7         args = validate_tool_args(tool.arg_model, call.arguments) # untrusted → validate
8         result = await tool.invoke(args) # YOUR code runs
9         messages.append(tool_result_message(call.id, result)) # feed back; loop again
```

In the loop (*Block 22*): the gateway call is *l.2*, the “final answer?” check is *l.3*, argument validation is *l.7* (the guardrail), *your* function runs on *l.8*, and its result is fed back on *l.9* before the loop repeats; *l.6* iterates *all* tool calls the model requested this turn (parallel tool calls).

A `Tool` = name + description + `arg_model` (Pydantic) + `fn`; its `spec()` emits the provider schema. The `tools` list is both the *menu* the model sees *and* the execution *allowlist* — a tool not in it is invisible and, if hallucinated, rejected by the registry.

The gateway surfaces tool requests on `Completion.tool_calls`: you send `tools=[...]`, the fine-tuned model emits a tool call in the trained format, and the provider parses it, sets `stop_reason="tool_use"`,

and populates `message.tool_calls` (LiteLLM normalises every provider's shape). The loop is bounded by `max_iterations` so a runaway model fails loudly.

9.1 Agents as tools

A tool's `fn` can be *another agent's run* — the supervisor / sub-agent pattern, with the coordinator model deciding at runtime which specialist to call. Each nested call rides the same gateway spine (retry/validation/cost/tracing), and errors propagate through the taxonomy (a `GuardrailError` is fed back for the model to correct; a terminal `AllModelsFailedError` bubbles up). Bound the number of calls with a shared budget (count, depth, or accumulated `cost_usd`); deep trees are where an engine's child workflows earn their place.

Block 23 ■ An agent wrapped as a tool

```
1 async def classify(args: ClassifyArgs) → str:
2     result = await sentiment_agent.run(TextIn(text=args.text)) # the sub-agent runs
3     return result.parsed.model_dump_json()
4
5 classify_tool = Tool(name="classify_sentiment", description="Classify sentiment.",
6                     arg_model=ClassifyArgs, fn=classify) # fn = another agent
```

Mechanism vs policy. The *loop* (`run_with_tools`) and the `Tool` interface are generic **package** machinery. The concrete tools (`get_weather`, `WeatherArgs`) are **project-specific** and defined in the repo, then passed in — the same dependency-injection split as `RouterLike` and `Step`.

10 eval/harness.py

Prompt and model changes are guesses until measured. The harness is a small loop: *dataset* → *run through a task* → *score* → *report* → *compare*.

Dataclass vs BaseModel, again. `EvalCase`, `Score`, `CaseResult`, `EvalReport` are **dataclasses** — internal records you author or the harness builds; no untrusted boundary to validate. The one `BaseModel` in the file is `JudgeVerdict` — the LLM judge's *output*, validated via `response_model` because *that* crosses the model boundary.

Block 24 ■ harness.py – run every case, concurrently and fault-isolated

```
1 async def run_eval(dataset, task, scorers, *, concurrency=8) → EvalReport:
2     semaphore = asyncio.Semaphore(concurrency) # bound width; big datasets don't flood
3     async def run_one(case):
4         async with semaphore: # acquire a slot; released on exit
5             try:
6                 output = await task(case.input) # the agent under test
7             except Exception as exc:
8                 return CaseResult(case_id=case.id, error=f"task: {exc}") # one bad case
9             scores = [await s(output, case) if isinstance(r := s(output, case)) else r
10                      for s in scorers] # sync or async scorers
11             return CaseResult(case_id=case.id, output=output, scores=scores)
12     return EvalReport(results=list(await asyncio.gather(*(run_one(c) for c in dataset))))
```

- **Task** = `async (input) -> output` (Block 24, l.6) — usually the agent under test, `lambda inp: agent.run(inp)`.
- **Scorer** = `(output, case) -> Score` — deterministic ones (`exact_match`, `numeric_close`) compare `output` to `case.expected`; `llm_judge` is reference-free, grading against a *rubric* through the gateway. Both are applied at Block 24, l.9.

- `compare(baseline, candidate)` — per-scorer mean deltas: the regression check, wired into CI to fail a PR that regresses a metric.

`async with semaphore` (*Block 24, l.4*; the semaphore is created on *l.2*) acquires one of `concurrency` permits, releasing it on exit even on error — the guaranteed-cleanup property of `with` — so `gather` (*l.12*) launches all cases but only N hit the LLM at once. A case that raises is captured as an errored `CaseResult` (*l.8*) rather than sinking the batch, and is surfaced in the summary, never silently averaged over.

Closing

One discipline runs through every module. A typed `Completion` makes “the output is validated or it is an exception” structural. A `@contextmanager` makes “open a span, always close it, record errors” structural. A per-key lock or a `WHERE` clause makes “commit exactly once” structural. A `Protocol` and a `Step` type make “depend on a shape, not a concretion” structural. In each case the guarantee lives in *one place* and the *mechanism* enforces it — not a convention every call site must remember. That is what makes the core small enough to hold in your head, and correct enough to build on.